Introduction to Web Development

- **Web Development** is the process of **creating websites or web applications** that can be accessed through the internet.
- It involves **designing, coding, and maintaining websites**.
- There are **two main parts**:
    1. **Frontend (Client-side)** – What users see and interact with on their browser (HTML, CSS, JavaScript).
    2. **Backend (Server-side)** – Handles data, storage, and business logic (Python, PHP, Java, Node.js, databases).
- Goal: Make websites **functional, interactive, and user-friendly**.

---

1. **Webpage:**
   A **webpage** is a single document that can contain **headings, paragraphs, images, links, videos, tables, forms, and other multimedia elements.**
   It is written using **HTML (HyperText Markup Language)** and can also use **CSS** and **JavaScript** for styling and interactivity.
   👉 Example: https://www.google.com/about is a single webpage.

2. **Website:**
   A **website** is a **collection of related webpages** that are connected through hyperlinks and usually share a common domain name.
   👉 Example: All webpages under https://www.google.com together form the **Google website**.

So in short:
📄 **Webpage** → single page (like a page in a book)
🌐 **Website** → collection of webpages (like the whole book)

A **website can be either static or dynamic** — it depends on **how** it's built and **how** the content is delivered.

Let's understand the difference clearly 👇

🧱 Static Website

- The content **does not change automatically**.

- Each webpage is an **HTML file** stored on the server.
- If you want to change something, you must **edit the code manually**.
- No database or server-side scripting is used.

Example:
A personal portfolio site with only index.html, about.html, and contact.html pages.

Technologies used:
HTML, CSS, JavaScript (only on the client side).

Example websites:

- Simple business websites
- Portfolio or resume sites
- Company info pages

⚙️ **Dynamic Website**

- The content **changes automatically** based on **user input, database, or server logic**.
- Webpages are generated **dynamically** using server-side languages.
- Uses a **database** (like MySQL, MongoDB, etc.) to store and retrieve content.

Example:
Facebook, Amazon, YouTube — content changes for each user.

Technologies used:
Python (Flask/Django), Java (Spring), PHP, Node.js, etc.

✅ **In short:**

Type	Content Changes	Uses Database	Example
Static Website	No	No	Portfolio, Company Info
Dynamic Website	Yes	Yes	Facebook, Amazon

1️⃣ **Static Webpage Example (HTML only)**

📄 File: index.html

```
<!DOCTYPE html>
<html>
<head>
 <title>Welcome Page</title>
</head>
<body>
 <h1>Welcome to My Website</h1>
 <p>Hello, my name is Jani Basha.</p>
 <p>Today's date is 6 October 2025.</p>
</body>
</html>
```

🧩 **Explanation:**

- The content (like your name and date) is **fixed**.
- If you want to change the date or text, you must **edit this file manually**.
- No backend or database is used.
  ✅ This is a **static website**.

⚙️2️⃣ **Dynamic Webpage Example (Using Flask in Python)**

📁 Folder structure:

```
my_flask_app/
 ├── app.py
 └── templates/
     └── index.html
```

🐍 **File: app.py**

```
from flask import Flask, render_template
from datetime import datetime

app = Flask(__name__)
```

```
@app.route('/')
def home():
    current_date = datetime.now().strftime("%d %B %Y, %A")
    user_name = "Jani Basha"
    return render_template('index.html', name=user_name, date=current_date)


if __name__ == '__main__':
    app.run(debug=True)
```

📄 **File: templates/index.html**

```
<!DOCTYPE html>
<html>
<head>
  <title>Dynamic Page</title>
</head>
<body>
  <h1>Welcome to My Dynamic Website</h1>
  <p>Hello, {{ name }}!</p>
  <p>Today's date is {{ date }}</p>
</body>
</html>
```

⚡ **Explanation:**

- Flask automatically **inserts data** (name, date) into the HTML file using {{ ... }} placeholders.
- Every time you refresh the page, the **date and time update automatically**.
  ✅ This is a **dynamic website** because it's generated by code running on a server.

🧠 **Summary**

Feature	Static Website	Dynamic Website
Built using	HTML, CSS	Flask, Django, Node.js, etc.
Content	Fixed	Changes automatically
Database	❌ No	✅ Yes (optional)
Example	Portfolio	Facebook, YouTube

What is the Web?

What is the Web (World Wide Web)?

- The **World Wide Web (WWW)** is a **system that allows access to a collection of websites and webpages over the Internet**.

> **Meaning of "over the Internet"**
> When we say:
> **"over the Internet"**
> it simply means:
> 👉 **using the Internet network**

- It enables users to **view and interact with information** using a **web browser**.
- **Websites are stored on web servers**, and **browsers request those websites using HTTP/HTTPS and display them**.
- The Web connects information using **URLs and hyperlinks**.

Example:

- www.google.com ⟶ a **website**
- Google home page (www.google.com) ⟶ a **webpage**

🧠 **Final one-line definition (Perfect for exams/interviews) (Follow this one):**

The World Wide Web is a system that allows users to access and interact with websites and webpages stored on web servers through the Internet using a browser.

A browser uses WWW rules to request a webpage, the Internet carries the request to a web server, the server responds using the same rules, and the browser renders the page for the user.

What is a Web Browser?

- A **Web Browser** is a **software application** that allows users to **access and view websites**.
- Popular browsers: **Google Chrome, Mozilla Firefox, Microsoft Edge, Safari**.
- **Function:**
    1. You type a website URL.
    2. Browser sends a request to the server.
    3. Browser receives the website data and displays it on your screen.

Easy Example: Browser is like a **TV**, and websites are **channels you watch**.

What is a Web Server?

- A **Web Server** is a **special computer** that stores website files (HTML, CSS, JS, images, etc.).
- **Function:** When a browser requests a website, the server **sends the website files back** to the browser.
- Examples of web servers: **Apache, Nginx, Microsoft IIS**.

Easy Example: Server = **Pizza shop**; Browser = **customer ordering pizza**; Server sends back pizza (website).

What is a Web Application?

- A **Web Application** is a **dynamic website** that allows users to **interact and perform tasks online**.
- Unlike a simple website, it **processes user input, communicates with a database**, and **provides personalized content**.

- Examples:
    - **Gmail** ⟶ Send and receive emails
    - **Facebook** ⟶ Post, like, and comment
    - **Amazon** ⟶ Buy products online

Key difference:

- **Website:** Shows information (static or simple content)
- **Web Application:** Allows user interaction and data processing (dynamic and interactive)

🌐 Website

- You can **only read or see information**.
- It is mostly **static** — content doesn't change unless the developer updates it.
- Example:
    - College website ⟶ shows info about courses and contact details.
    - News website ⟶ you only read the news.

👉 So yes, a **website is read-only or view-only.**

⚙️ Web Application

- You can **interact** with it — give input, click buttons, log in, upload files, buy products, etc.
- It is **dynamic** — content changes based on user or data.
- Example:
    - Gmail ⟶ you can send or delete emails.
    - Amazon ⟶ you can search, buy, add to cart.

👉 So yes, a **web application is used for interaction.**

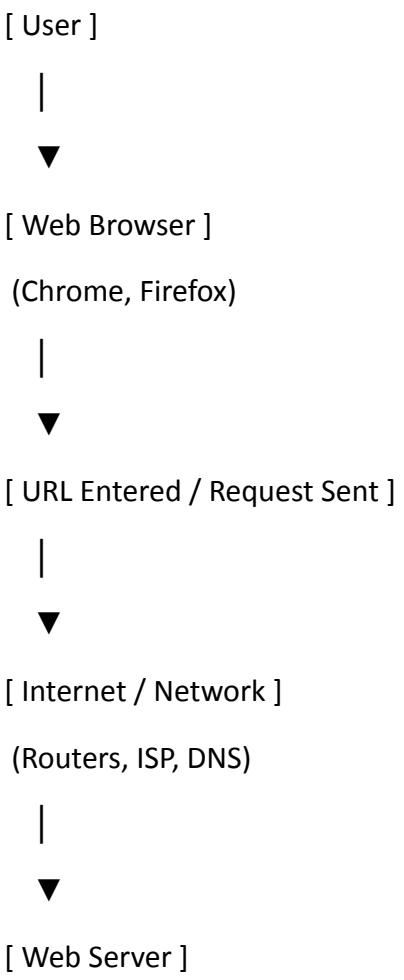
✅ In one line:

A **website** is for **reading information**,
A **web application** is for **doing actions or interactions**.

✅ Summary Table for Students

Topic	Simple Explanation
Web Development	Creating websites and web applications using frontend & backend technologies.
Web	Collection of websites and webpages accessible via internet.
Web Browser	Software to access websites (Chrome, Firefox).
Web Server	Computer that stores website files and sends them to browsers.
Web Application	Interactive website that processes data and allows user actions online.

Flowchart: How Web Works



(Stores Website Files)

|

▼

[Web Application / Website]

(Dynamic or Static Content)

|

▼

[Browser Displays Page to User]

Explanation in Simple Words

1. **User** → Wants to visit a website.
2. **Web Browser** → Software to access websites.
3. **URL / Request** → Browser asks the website by sending a request.
4. **Internet / Network** → Request travels through the internet.
5. **Web Server** → Special computer where the website or web application is stored.
6. **Web Application / Website** → Server processes request, sends files or dynamic content back.
7. **Browser Displays Page** → User sees the website and can interact with it.

✅ **Memory Trick for Students**

User → Browser → Internet → Server → Website → Browser

- Think: **"I ask → Internet carries → Server responds → I see it"**

🌐 **What is the Internet?**

The **internet** is a huge network of computers all around the world that are **connected** to share information.
It's like a **giant road system** that connects cities, but instead of cars, it carries **data (text, images, videos, etc.)**.

⚡ **How Internet Works (Step by Step in Simple Words)**

1. **You type a website name (URL)** in your browser (example: www.google.com).
2. The request first goes to your **Internet Service Provider (ISP)** (like Jio, Airtel, BSNL, etc.).
    - ISP = The company that gives you internet connection.
3. ISP asks the **DNS server (phonebook of internet)** to find the IP address of the website.
    - Example: www.google.com → 142.250.183.78.
4. Once the IP address is found, your request travels through the **internet network (routers, cables, satellites, etc.)** to reach the website's **server**.
    - Server = A special computer where the website is stored.
5. The **server sends back the webpage data** (text, images, videos).
6. Your **browser shows the page** on your screen.

📊 **Flow (Simple Diagram in Text)**

[You (Browser)]

|

▼

[ISP (Internet Provider)]

|

▼

[DNS Server (Finds IP Address)]

|

▼

[Website Server (Sends Data)]

|

▼

[Your Browser Displays Page]

✅ Real-Life Example

- Imagine you want to order a pizza 🍕.
    - **Browser** = You calling the pizza shop.
    - **ISP** = Your telephone service (Jio, Airtel).
    - **DNS** = Phonebook that gives you the shop's number.
    - **Server** = Pizza shop making the pizza.
    - **Internet cables/routers** = Roads that deliver the pizza.
    - **Browser again** = You receiving and eating the pizza 🍕.

🌐 Internet in Simple Words

- **Internet** = A big network that connects computers all over the world.
- It allows us to **share and access information** (websites, emails, videos, etc.).

⚡ How Internet Works (Step by Step)

1. **You type a website name** in a browser (like www.google.com).
2. The request goes to your **Internet Service Provider (ISP)** (like Jio, Airtel).
3. ISP asks the **DNS server** (like a phonebook) to find the website's **IP address**.
4. The request is sent through the **internet network** (routers, cables, satellites).
5. It reaches the **website's server** (a computer that stores the site).
6. The **server sends back the webpage data**.
7. Your **browser shows the page** on your screen.

✅ Easy Example

- **Browser** = You
- **ISP** = Your internet provider (Airtel, Jio)
- **DNS** = Phonebook of internet (finds addresses)
- **Server** = Shop where the website lives

- **Internet cables/routers** = Roads carrying the data
- **Page on screen** = Pizza delivered to you 🍕

👉 **In short:**
Browser → ISP → DNS → Server → Browser

👉 https://www.google.com/about
in a **very simple and clear way**, step by step.

🔗 **Full URL**

https://www.google.com/about

This is a **webpage URL**. Each part has a meaning.

1️⃣**https:// → Protocol**

- It stands for **HyperText Transfer Protocol Secure**.
- It tells the browser **how to communicate** with the server.
- https means the data is **encrypted and secure** 🔒.

👉 Example:

- http:// → not secure
- https:// → secure (used by almost all modern websites)

2️⃣**www → Subdomain**

- www means **World Wide Web**.
- It is a **subdomain** of google.com.
- Many websites use it, but it's **optional**.

👉 google.com and www.google.com usually point to the same site.

3️⃣ **google.com → Domain Name**

- This is the **main website name**.
- It uniquely identifies Google on the Internet.
- Behind the scenes, this maps to Google's **server IP address**.

👉 Example:

- google.com
- amazon.com
- facebook.com

4️⃣ **/about → Path / Webpage**

- This tells the server **which page** the user wants.
- /about means the **About Google webpage**.
- It is **one webpage inside the Google website**.

👉 Other examples:

- /careers
- /contact
- /products

🧠 **What happens internally?**

1. You type the URL in the browser.
2. Browser contacts **Google's server** using https.
3. Server processes the request.
4. Server sends back the **About page HTML**.
5. Browser displays the webpage.

📌 **Important Understanding (This clears confusion)**

- ✅ **google.com** → Website
- ✅ **/about** → One webpage inside that website

- ✅ It runs on **Google's servers**
- ✅ It is a **website page**, not a web application
  (because you are mainly **reading information**, not performing actions)

🧩 **One-line explanation:**

https://www.google.com/about is a secure URL that opens the *About* webpage of the Google website, hosted on Google's servers.

🔹 **"How does the browser communicate with the server?"**

The browser and server **talk to each other using rules**.
These rules are called a **protocol**.

👉 In your example, the protocol is **HTTP / HTTPS**.

🧠 **Simple real-life example**

Think like this:

- 📞 **Protocol** = language/rules of talking
- 🧑 Browser = customer
- 🖥️ Server = shop

If both don't follow the same rules, **communication fails**.

🌐 **Actual Communication Steps (Very Simple)**

1️⃣ **You enter a URL**

https://www.google.com/about

The browser understands:

- Use **HTTPS**
- Contact **google.com**
- Ask for the **/about page**

2️⃣ Browser creates a request (HTTP Request)

The browser sends a message like:

“Hello Google server 👋
I want the /about page.
I am using HTTPS.”

This is called an **HTTP Request**.

3️⃣ Server receives the request

Google’s server:

- Understands HTTPS rules
- Finds the /about page
- Prepares the response

4️⃣ Server sends a response (HTTP Response)

The server replies:

“Here is the /about page.
Here is the HTML, CSS, images.”

This is called an **HTTP Response**.

5️⃣ Browser displays the page

- Browser reads the HTML
- Loads CSS, images, JS
- Shows the webpage on your screen

🔁 Communication Flow (Easy Diagram)

Browser ——(HTTP Request)——▶ Server

Browser ◀—(HTTP Response)—— Server

🔒 Why HTTPS?

HTTPS means:

- Data is **encrypted**
- No one can read or change it in between
- Safe for passwords, logins, payments

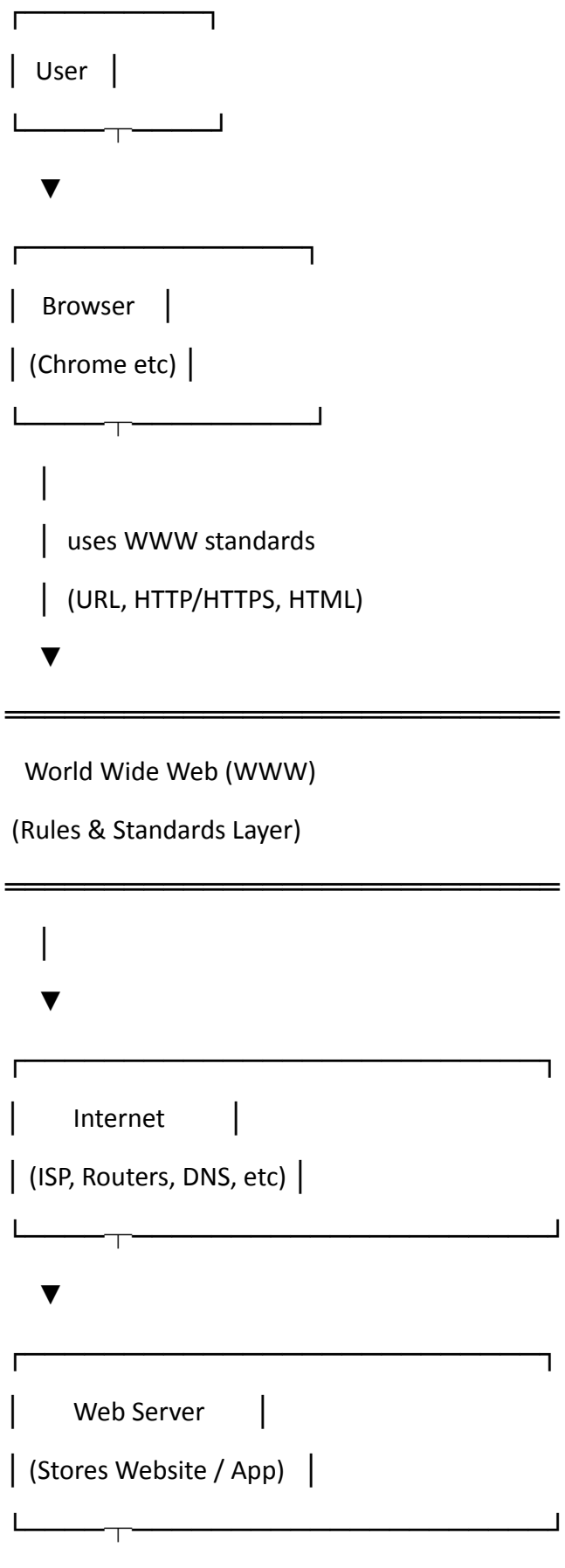
📌 Very Important Line (Remember This)

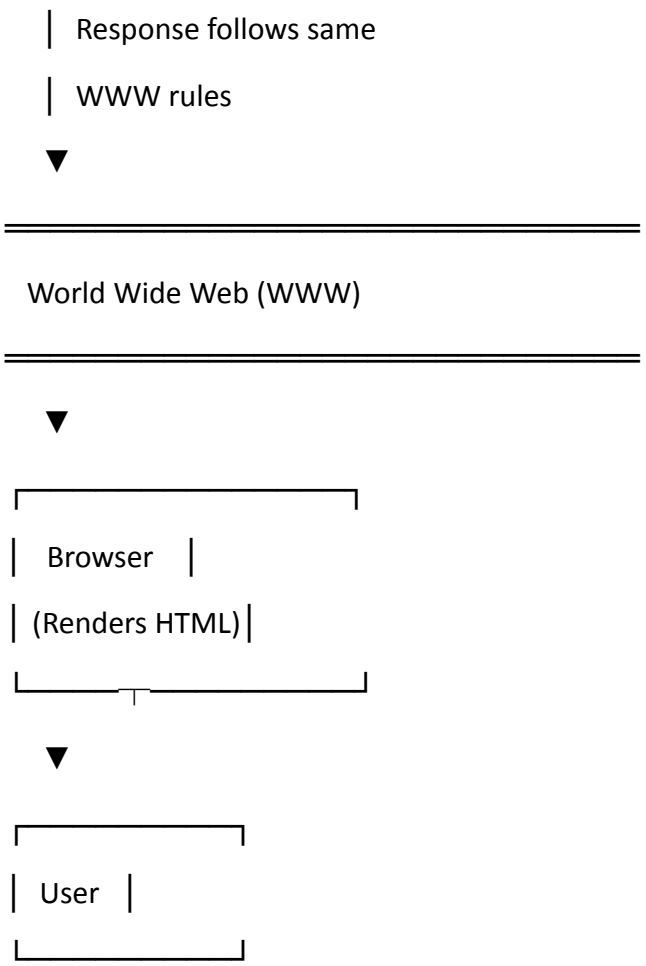
The browser communicates with the server by **sending HTTP requests and receiving HTTP responses**, following HTTP/HTTPS rules.

🧩 One-line summary:

- **Protocol (HTTP/HTTPS)** → defines *how* browser and server talk
- **Browser** → requests data
- **Server** → responds with data

🌐 Web Request–Response Flow (Correct View)


User
Browser
(Chrome etc)
uses WWW standards
(URL, HTTP/HTTPS, HTML)
World Wide Web (WWW)
(Rules & Standards Layer)
Internet
(ISP, Routers, DNS, etc)
Web Server
(Stores Website / App)



🔑 **Key Understanding (MOST IMPORTANT)**

- **WWW is NOT a machine**
- **WWW is NOT a server**
- **WWW is NOT the Internet**

👉 **WWW is a RULES layer** used by:

- Browser to create requests
- Server to send responses

🧠 **One-line clarity**

The browser and web server communicate **using WWW standards**, while the **Internet only carries the data**.

🔥 **Final memory trick**

Layer	Role
Internet	Carries data
WWW	Defines how web data is accessed
Web Server	Stores website
Browser	Displays website

What is WSGI?

WSGI stands for Web Server Gateway Interface.

“WSGI stands for Web Server Gateway Interface. It is a Python standard that defines how a web server talks to a Python web application or framework, acting as a bridge between them.”